

Documentation TankOS1

Groupe Spécialité Sciences de l'Ingénieur
16 Septembre 2026

Firmware de contrôle pour plateforme chenillée radiocommandée

Projet Tank Terminale, Spécialité Sciences de l'Ingénieur, session 2026/2027 — Version 1 — Carte cible Arduino Mega 2560 (ATmega2560)

Sommaire

Introduction	1
1. Présentation générale	1
2. Architecture matérielle	2
3. Architecture logicielle et ordonnancement FreeRTOS	3
4. Module Traction	4

Introduction

1. Présentation générale

TankOS est le firmware embarqué pilotant une plateforme chenillée radiocommandée à deux moteurs de traction et une tourelle motorisée équipée d'un système d'armement simulé (ESC double). Le firmware est écrit en C++ pour l'environnement Arduino, compilé et déployé via PlatformIO, et s'exécute sur une carte Arduino Mega 2560 basée sur le microcontrôleur ATmega2560 (8 bits, 16 MHz, 8 Ko de SRAM, 256 Ko de mémoire programme).

Le système repose sur un noyau temps réel FreeRTOS (portage AVR de la bibliothèque feilipu/FreeRTOS) et organise le traitement en cinq tâches concurrentes ordonnancées par priorité. Cette architecture remplace une structure séquentielle classique de type `loop()` et garantit que les fonctions critiques de sécurité (perte de liaison radio, surintensité, décharge batterie) sont exécutées avec une latence bornée, indépendamment de la charge des autres sous-systèmes.

Les fonctions principales assurées par le firmware sont les suivantes :

- Pilotage différentiel de la traction par relais et potentiomètres numériques SPI
- Pilotage de la tourelle : élévation et rotation par servomoteurs, activation de deux contrôleurs de vitesse électroniques (ESC)
- Réception de dix voies de la radiocommande par capture d'impulsions PWM en interruption
- Supervision de deux batteries (châssis et tourelle) et d'un capteur de courant, avec bridage automatique de la vitesse et alertes
- Coupure matérielle de l'alimentation moteur en cas de défaillance (MOSFET de puissance piloté logiquement)
- Contrôle d'accès par badge RFID pour déverrouiller les commandes sensibles
- Interface de commande textuelle disponible simultanément sur liaison série USB et sur module Bluetooth HM-10
- Régulation thermique par ventilateur piloté en PWM matériel haute fréquence

2. Architecture matérielle

L'ensemble du brochage est centralisé dans le fichier `config.h`, seul point de référence pour toute modification physique du câblage. Les sous-systèmes matériels sont décrits ci-dessous ; le détail exhaustif des broches figure en annexe.

2.1 Chaîne de traction

Chaque moteur de traction est commandé par un ensemble de relais montés sur une carte pilote MDRVCT60A002M. La direction (avant/arrière) de chaque moteur est sélectionnée par une paire de relais dédiés, et deux relais supplémentaires assurent la coupure générale de l'alimentation de chaque côté (fonction de sécurité indépendante de la direction). La vitesse est réglée par un potentiomètre numérique SPI MCP41100 par moteur, adressé par une broche CS dédiée (42 pour le moteur gauche, 43 pour le moteur droit), le bus SPI (SCK/MOSI/MISO sur les broches matérielles 52/51/50) étant partagé avec le lecteur RFID.

2.2 Tourelle

La tourelle intègre trois servomoteurs analogiques pour le mouvement d'élévation et la rotation (broches 2, 3 et 6 pilotées en parallèle) et deux ESC pour l'armement (broches 4 et 5, pilotage en largeur d'impulsion de 1000 à 2000 microsecondes).

2.3 Réception radiocommande

Une radiocommande de type huit voies transmet des signaux PWM captés par interruption logicielle sur changement d'état (bibliothèque `enableInterrupt`). Les voies exploitées couvrent la direction et la vitesse de traction, l'orientation et l'élévation de la tourelle, l'armement, la vitesse de rampe (VRB) et deux interrupteurs auxiliaires (SWC, SWD). La capture est réalisée dans `ISR.cpp`, fichier isolé du reste du firmware car il gère les temporisations critiques en contexte d'interruption.

2.4 Lecteur RFID

Le contrôle d'accès admin repose sur un lecteur RC522 en liaison SPI, avec sélection sur la broche 41 et réinitialisation sur la broche 40. Sur Mega, le bus SPI matériel est situé sur les broches 50/51/52 (contrairement à un Uno où il se trouve sur 11/12/13), et des résistances série de 330 ohms sont insérées sur SCK et MOSI pour la compatibilité de niveaux logiques 5V du module.

2.5 Liaison Bluetooth

Un module HM-10 est connecté sur `Serial1` à 9600 bauds et fournit une interface de commande sans fil identique à la liaison série USB. Une broche BRK (37) permet de forcer une rupture de connexion Bluetooth de façon logicielle.

2.6 Capteurs et actionneurs auxiliaires

La température est relevée par un capteur I2C MCP9808. Deux ponts diviseurs analogiques mesurent la tension des batteries châssis (A1) et tourelle (A2), un capteur à effet Hall (type ACS) mesure le courant consommé (A0). Un ventilateur est commandé en modulation de largeur d'impulsion matérielle à 25 kHz par configuration directe du timer 2 (broche 9). Une LED RGB de statut (broches 46 à 49) restitue l'état global du système.

2.7 Sécurité matérielle

Un transistor MOSFET canal P (FQP27P06) commandé sur la broche 32 permet la coupure matérielle de l'alimentation de puissance de la chaîne de traction, indépendamment de l'état des relais de direction, en complément de la logique de sécurité logicielle.

3. Architecture logicielle et ordonnancement FreeRTOS

Le firmware utilise le portage AVR de FreeRTOS (feilipu/FreeRTOS) afin de garantir un ordonnancement préemptif par priorité. Cinq tâches sont créées au démarrage, chacune encapsulant une boucle infinie rythmée par `vTaskDelay()`, ce qui libère le processeur pour les tâches de priorité inférieure ou égale pendant l'attente.

Tâche	Priorité	Période	Responsabilité
TaskSafety	4 (la plus haute)	20 ms	Détection de perte de liaison radio, calcul du facteur de bridage batterie, coupure de l'alimentation traction, gestion de la LED de statut
TaskTraction	3	34 ms	Lecture des voies radio et application des consignes de vitesse/direction à la chaîne de traction
TaskTourelle	3	34 ms	Élévation, rotation et armement de la tourelle
TaskCommunication	2	20 ms	Traitement des commandes série/Bluetooth, gestion des ventilateurs, télémétrie périodique
TaskAdmin	1 (la plus basse)	50 ms	Scrutation du lecteur RFID et gestion du verrouillage admin

La priorité de TaskSafety est volontairement maximale : elle doit pouvoir préempter toute autre tâche afin de garantir la coupure immédiate de la traction et de la tourelle en cas de perte de signal, quelle que soit la charge des autres modules.

3.1 Synchronisation par mutex

Deux sémaphores mutex, déclarés globalement dans `config.h` et instanciés dans `main.cpp`, protègent les ressources partagées entre tâches :

- `spiMutex` : protège l'accès exclusif au bus SPI matériel, partagé entre le lecteur RC522 (`admin.cpp`) et les potentiomètres numériques MCP41100 (`traction.cpp`). La totalité d'une transaction (sélection du composant, transfert des octets, désélection) est encadrée par la prise et la libération du mutex, afin de garantir son atomicité.
- `stateMutex` : protège l'état partagé calculé par TaskSafety (indicateur de failsafe actif, facteur de bridage de vitesse) et consommé par TaskTraction et TaskTourelle.

Les prises de mutex sont systématiquement bornées dans le temps (`pdMS_TO_TICKS(5)`) : en cas d'indisponibilité, la tâche appelante renonce à l'opération pour le cycle courant plutôt que de bloquer indéfiniment, préservant ainsi le respect des échéances temporelles de l'ordonnanceur.

3.2 Communication inter-tâches

Les données de la radiocommande sont exposées par `getRecepteurData()`, qui recopie sous section critique (`taskENTER_CRITICAL()` / `taskEXIT_CRITICAL()`) les variables volatiles renseignées par les routines d'interruption vers une structure `RcData` locale à chaque tâche appelante. Cette structure regroupe les huit canaux : axes de traction, axes de tourelle, interrupteurs SWC/SWD, potentiomètres VRA/VRB.

3.3 Point d'entrée et supervision

La fonction `setup()` initialise séquentiellement chaque module matériel, crée les mutex, instancie les cinq tâches via `xTaskCreate()` en vérifiant systématiquement leur code de retour, puis démarre l'ordonnanceur via `vTaskStartScheduler()`. La fonction `loop()` Arduino reste vide, le contrôle étant entièrement cédé au noyau FreeRTOS. Un hook `vApplicationStackOverflowHook()` est enregistré pour signaler sur le port série tout dépassement de pile d'une tâche et figer volontairement le système, ce comportement étant préférable à une corruption mémoire silencieuse.

4. Module Traction

Le module `traction.cpp` implémente une machine à états par moteur (structure `Motor`), avec trois états : `IDLE` (moteur arrêté), `RUNNING` (moteur en mouvement dans une direction donnée) et `SWITCHING` (phase transitoire lors d'une inversion de direction). Un changement direct de sens de marche sans passage par un arrêt intermédiaire de 200 millisecondes est proscrit afin de protéger l'électronique de puissance et les moteurs des courants d'appel liés à une inversion brutale.